Assignment 2

October 15, 2026

Assignment 2 Name : Ujwal Tikhe Roll No. : 135

[ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[2]:
```
df = pd.read_csv('emails.csv')
df.head()
```

[2]:

	Email No.	the	to	ect	and	for	of	a	you	hou	…	connevey	jay	\
0	Email 1	0	0	1	0	0	0	2	0	0	…	0	0	
1	Email 2	8	13	24	6	6	2	102	1	27	…	0	0	
2	Email 3	0	0	1	0	0	0	8	0	0	…	0	0	
3	Email 4	0	5	22	0	5	1	51	2	10	…	0	0	
4	Email 5	7	6	17	1	5	2	57	0	9	…	0	0	

	valued	lay	infrastructure	military	allowing	ff	dry	Prediction
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0	0
2	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0
4	0	0	0	0	0	1	0	0

[5 rows x 3002 columns]

[3]:
```
df.isnull().sum()
```

[3]:
```
Email No.     0
the           0
to            0
ect           0
and           0
             ..
military      0
allowing      0
ff            0
dry           0
Prediction    0
```

```
Length: 3002, dtype: int64
```

[4]:
```
df.dropna(how='any',inplace=True)
```

[5]:
```
x = df.iloc[:,1:-1].values
y = df.iloc[:,-1].values
```

[6]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.
↪25,random_state=10)
```

[7]:
```
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    PrecisionRecallDisplay,
    RocCurveDisplay
)

def report(classifier):
    y_pred = classifier.predict(x_test)

    cm = confusion_matrix(y_test, y_pred)

    display = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=classifier.classes_
    )
    display.plot()

    print(f"Accuracy:  {accuracy_score(y_test, y_pred)}")
    print(f"Precision Score:  {precision_score(y_test, y_pred)}")
    print(f"Recall Score:  {recall_score(y_test, y_pred)}")

    PrecisionRecallDisplay.from_estimator(
        classifier, x_test, y_test
    ).plot()

    RocCurveDisplay.from_estimator(
        classifier, x_test, y_test
    ).plot()
```

0.0.1 K-Nearest Neighbours Classifier

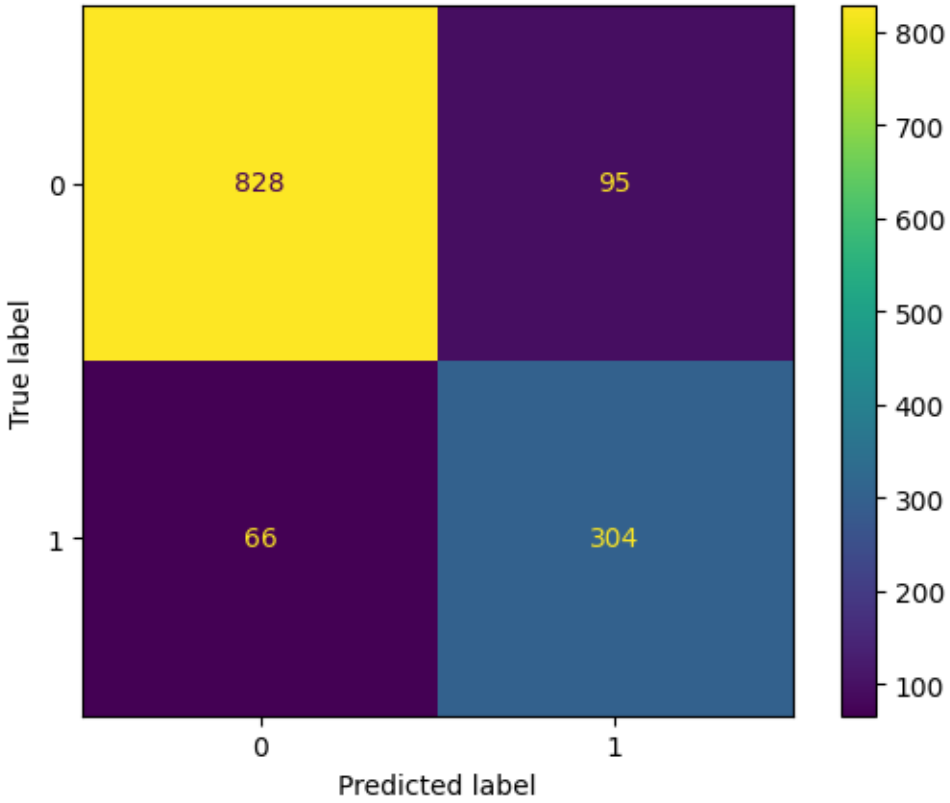
[8]:
```
from sklearn.neighbors import KNeighborsClassifier
```

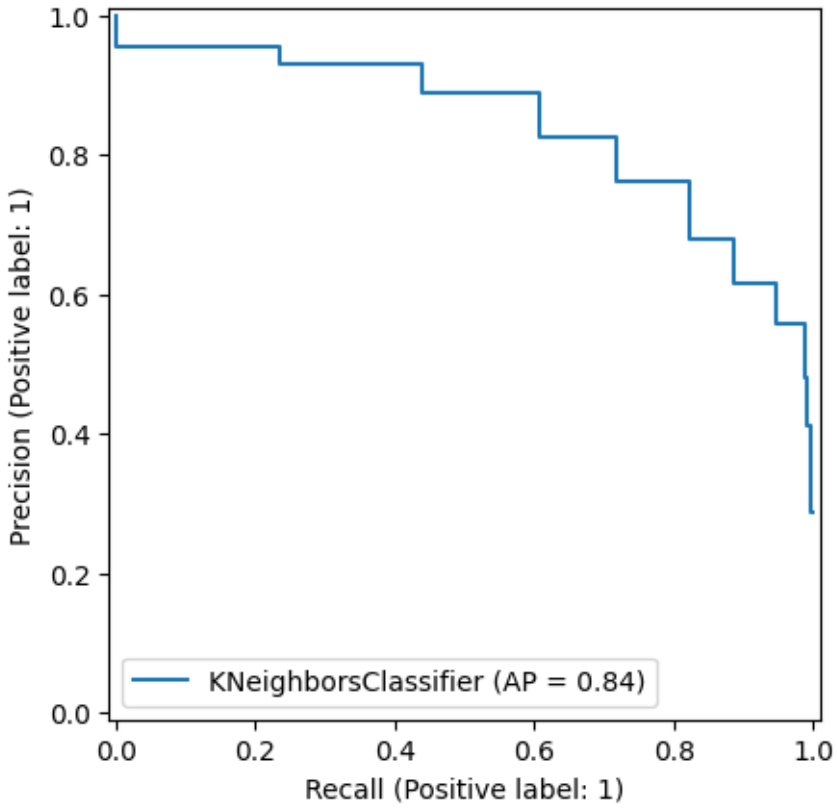
[9]:
```
kNN = KNeighborsClassifier(n_neighbors=10)
kNN.fit(x_train,y_train)
```

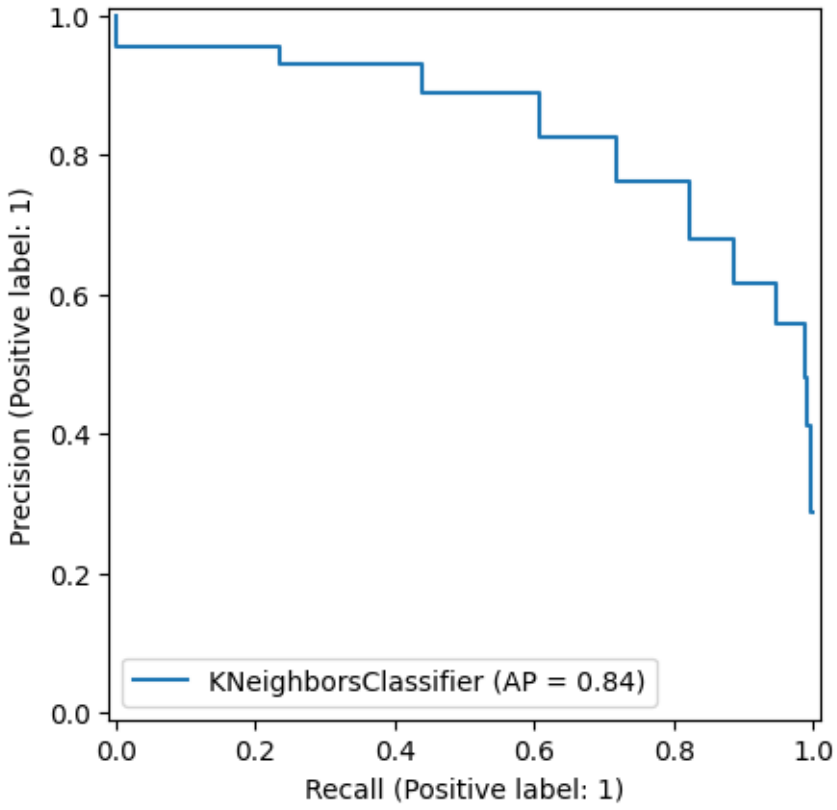
[9]:
```
KNeighborsClassifier(n_neighbors=10)
```

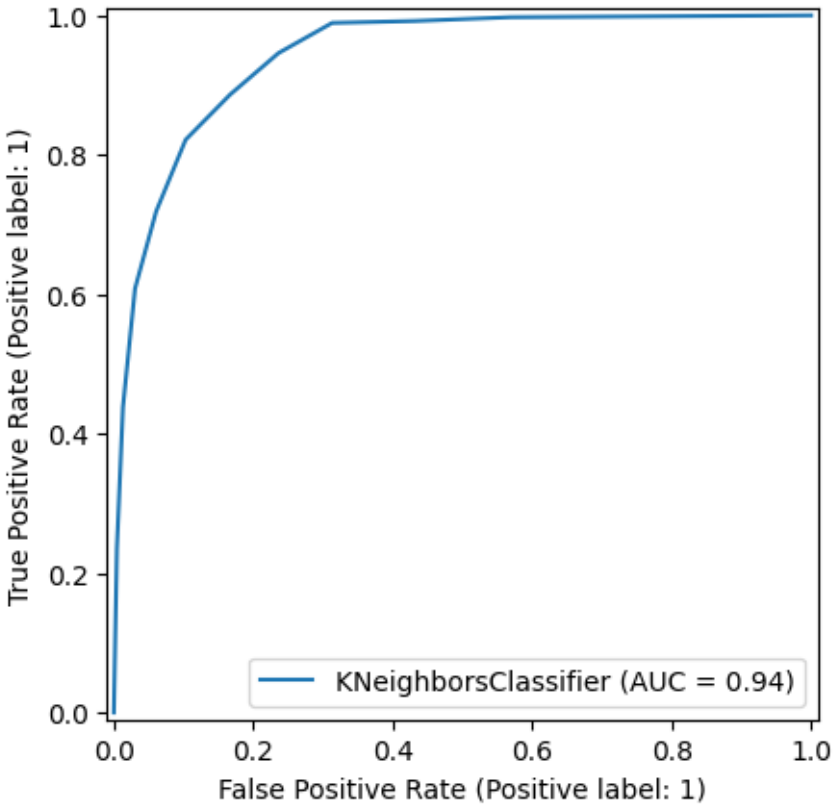
[10]:
```
report(kNN)
```

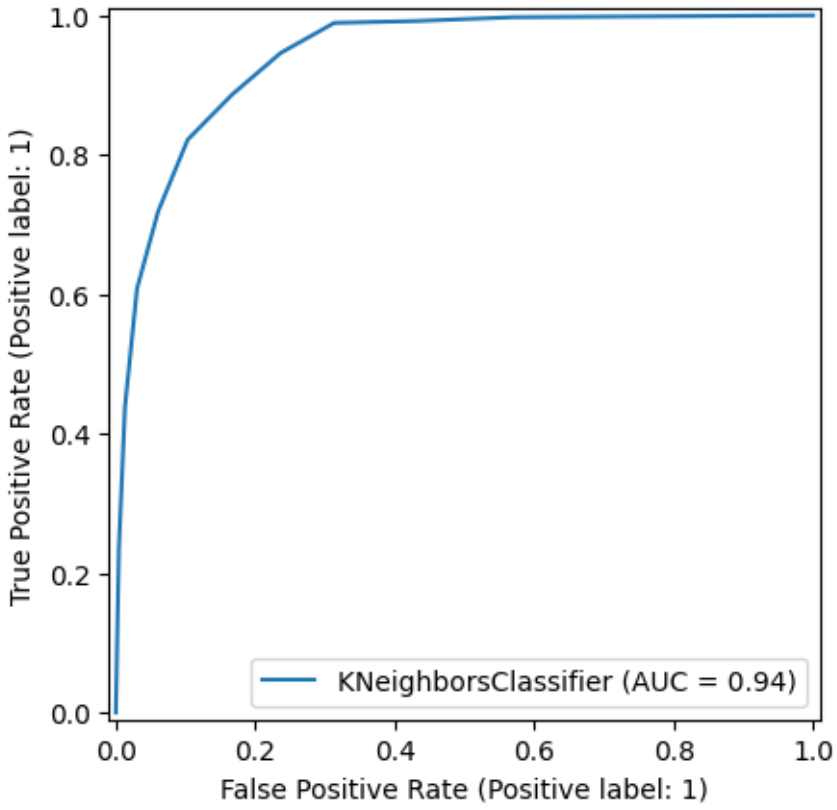
```
Accuracy:  0.8754833720030936
Precision Score:  0.7619047619047619
Recall Score:  0.8216216216216217
```




1.0
0.8
0.6
0.4
0.2
0.0
Precision (Positive label: 1)
KNeighborsClassifier (AP = 0.84)
0.0
0.2
0.4
0.6
0.8
1.0
Recall (Positive label: 1)


1.0
0.8
0.6
0.4
0.2
0.0
Precision (Positive label: 1)
KNeighborsClassifier (AP = 0.84)
0.0
0.2
0.4
0.6
0.8
1.0
Recall (Positive label: 1)


1.0
0.8
0.6
0.4
0.2
0.0
True Positive Rate (Positive label: 1)
KNeighborsClassifier (AUC = 0.94)
0.0
0.2
0.4
0.6
0.8
1.0
False Positive Rate (Positive label: 1)

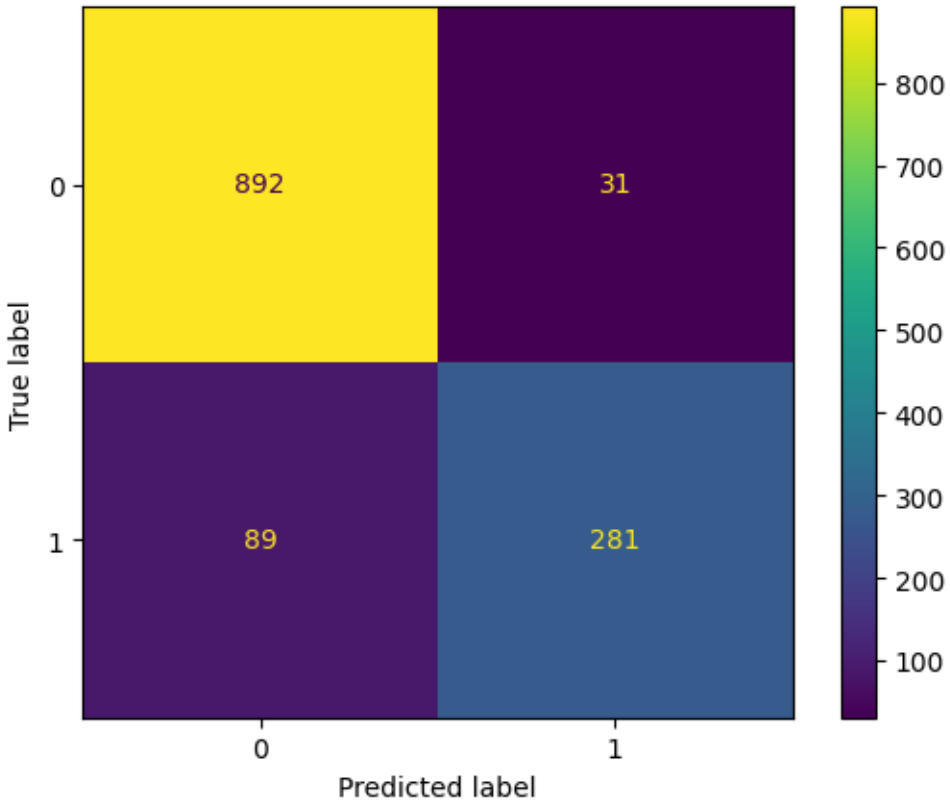


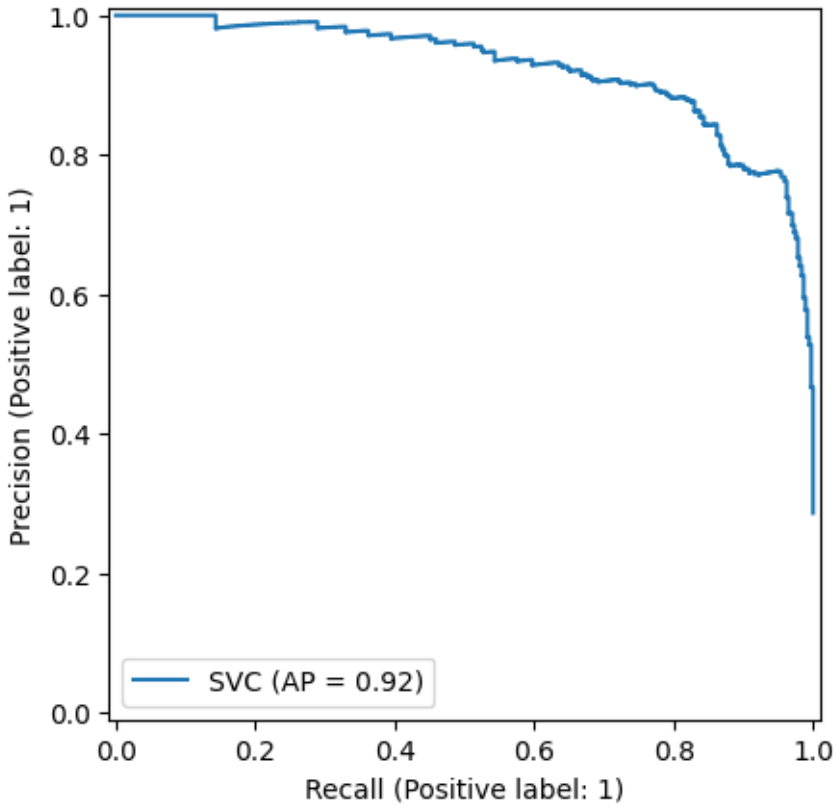
[11]:
```
from sklearn.svm import SVC
svm = SVC(gamma='auto',random_state=10)
svm.fit(x_train,y_train)
```

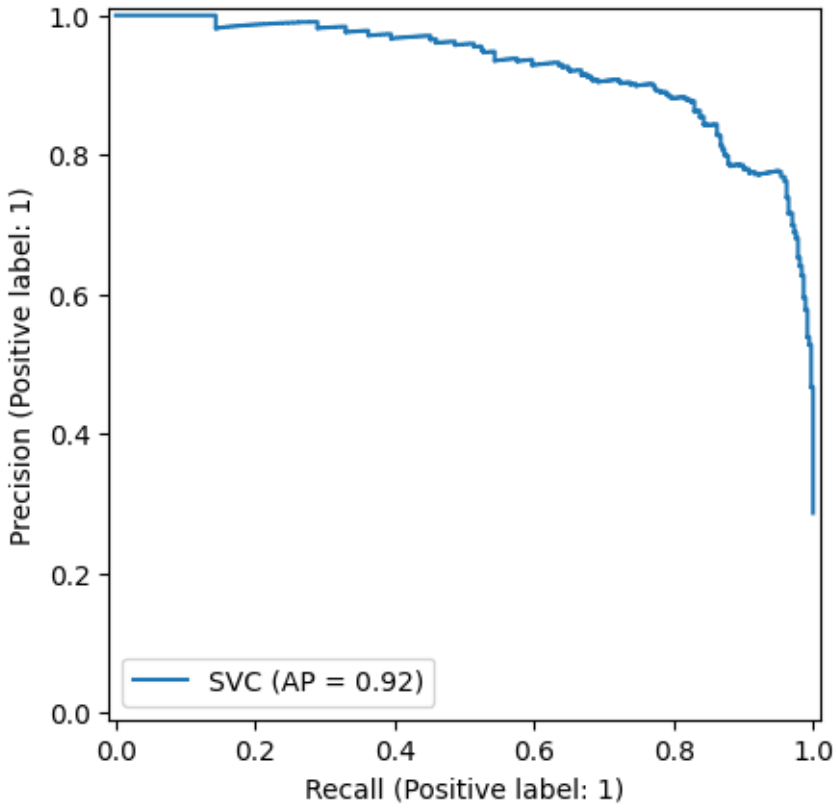
[11]:
```
SVC(gamma='auto', random_state=10)
```

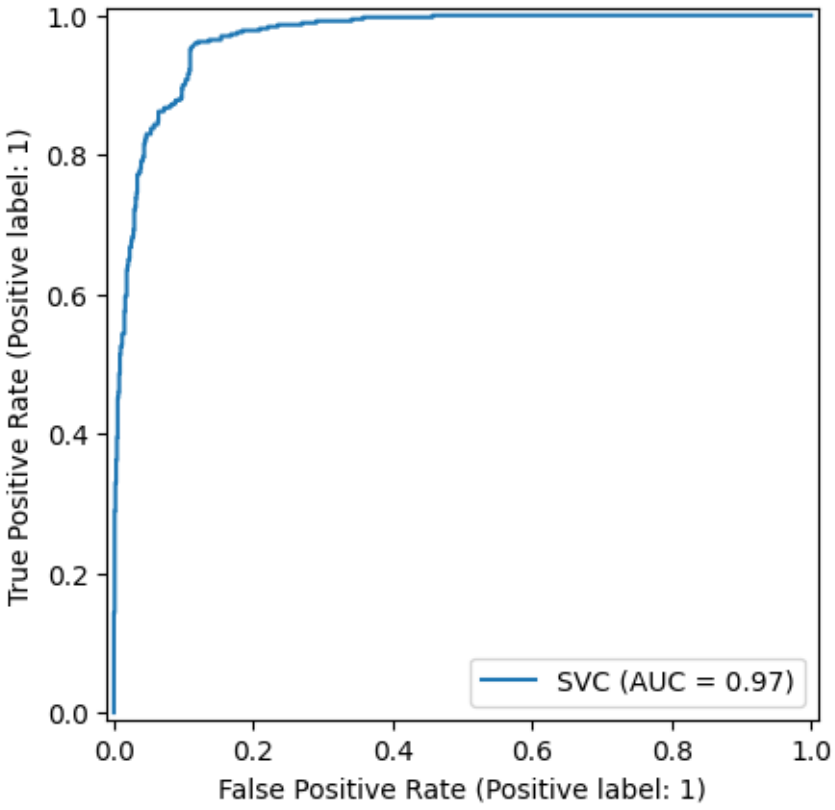
[12]:
```
report(svm)
```

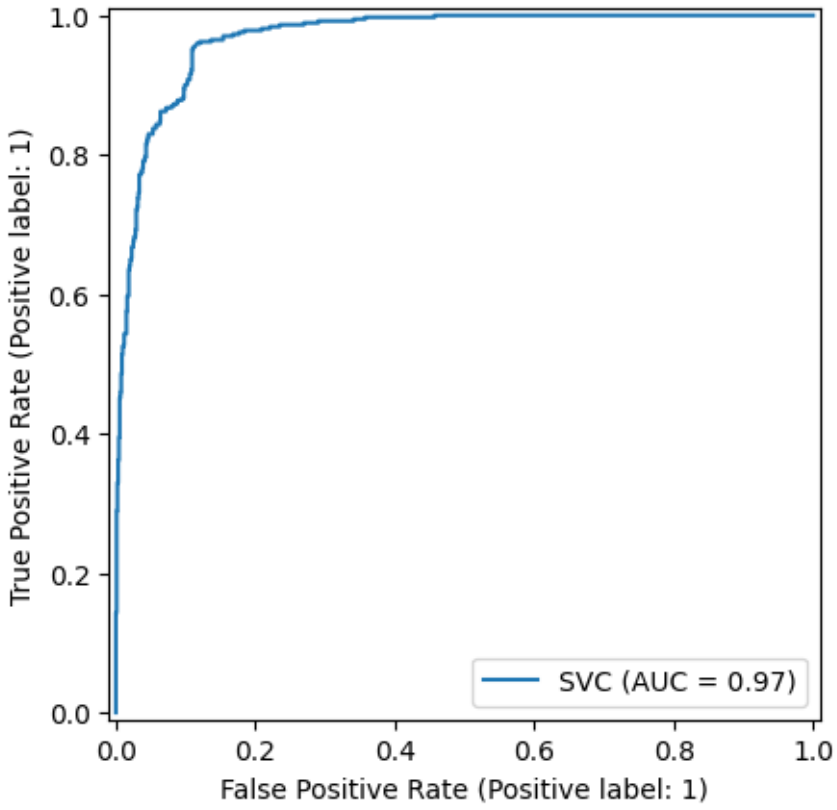
```
Accuracy:  0.9071925754060325
Precision Score:  0.9006410256410257
Recall Score:  0.7594594594594595
```


892
31
89
281
0
1
True label
0
1
Predicted label
800
700
600
500
400
300
200
100


1.0
0.8
0.6
0.4
0.2
0.0
Precision (Positive label: 1)
SVC (AP = 0.92)
0.0
0.2
0.4
0.6
0.8
1.0
Recall (Positive label: 1)


1.0
0.8
0.6
0.4
0.2
0.0
Precision (Positive label: 1)
SVC (AP = 0.92)
0.0
0.2
0.4
0.6
0.8
1.0
Recall (Positive label: 1)


1.0
0.8
0.6
0.4
0.2
0.0
True Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
False Positive Rate (Positive label: 1)
SVC (AUC = 0.97)


1.0
0.8
0.6
0.4
0.2
0.0
True Positive Rate (Positive label: 1)
SVC (AUC = 0.97)
0.0
0.2
0.4
0.6
0.8
1.0
False Positive Rate (Positive label: 1)

[ ]: